**Shiv Nadar University Chennai**

**CS3807 – Deep Learning Laboratory**

**Name** : M Kavya

**Reg. No** : 24011101061

## Experiment 7

**End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders**

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

# 1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

# 2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

# 3. Dataset and Experimental Setup

**Dataset: MNIST Handwritten Digit Dataset**

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

`https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist`

**Recommended laboratory subset:**

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.
The labels are not required as reconstruction targets. The original image is the target:

$$\boxed{x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where $x$ is the input image and $\hat{x}$ is its reconstruction.

# 4. Expected Input and Output

## Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

## Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

```
Original image
Reconstructed image
Reconstruction error
```

For the denoising model, display:

```
Original clean image
Noisy image
Denoised reconstruction
```

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

## 5. Overall Experimental Pipeline

The complete experiment follows:




The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

## 6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.
The encoder maps the input to a lower-dimensional representation:

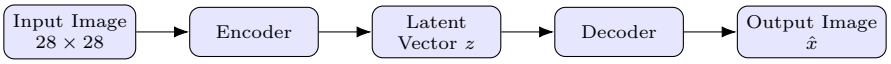
$$z = f_\theta(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_\phi(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2.$$



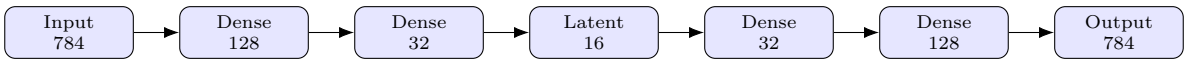

The latent vector is a compressed representation of the input.

## 7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.
Use the following architecture:




Use ReLU activation in hidden layers and sigmoid activation in the output layer.
**Suggested configuration:**

| Parameter | Value |
|---|---|
| Latent dimension | 16 |
| Optimizer | Adam |
| Learning rate | $10^{-3}$ |
| Batch size | 128 |
| Epochs | 20 |
| Loss | Binary Cross-Entropy |

# 8. Required Analysis of the Basic Autoencoder

**Plot 1: Original versus Reconstructed Images**

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

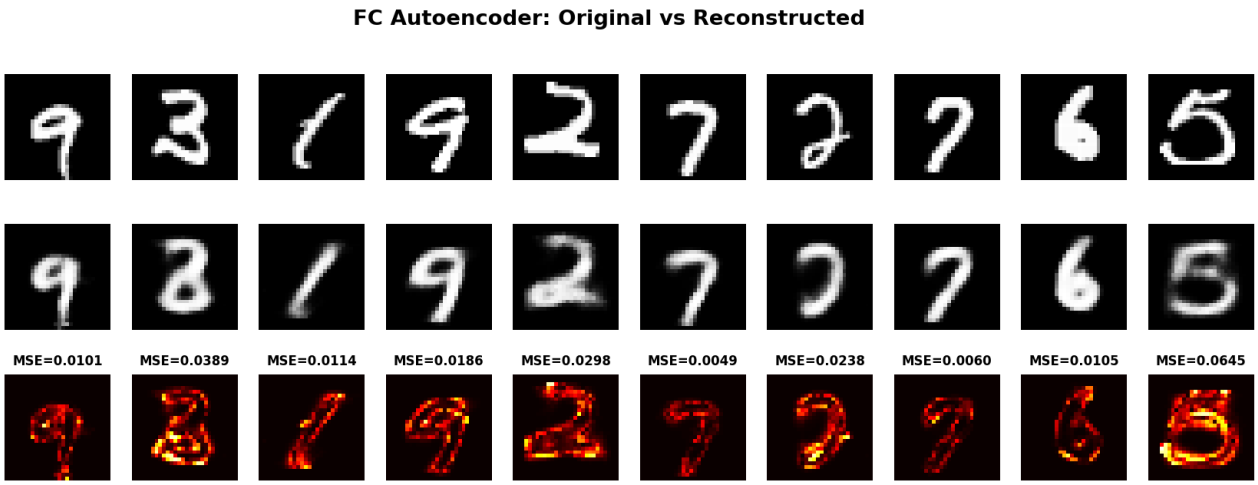



Figure 1: Plot 1: Original (top), FC-AE reconstruction (middle) and per-image reconstruction error map with MSE (bottom) for ten test images.

**Observation:** The fully connected autoencoder preserves the overall digit shape (9, 1, 7 and 6 are clearly recognisable), but thin strokes are lost and the edges are blurred. Easy samples with simple, thick strokes like 7 have low error, whereas 5 3 and 2 are the hardest; the errors are concentrated along stroke boundaries.

**Plot 2: Training and Validation Reconstruction Loss**

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

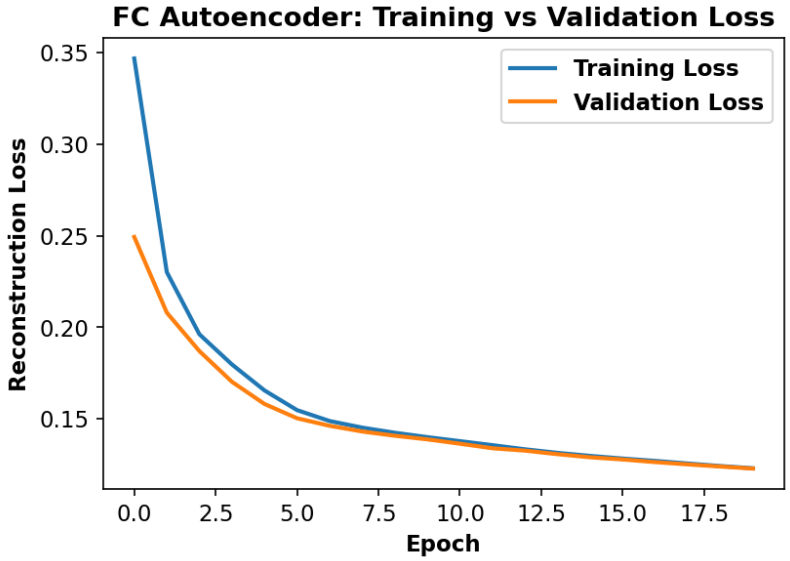



Figure 2: Plot 2: Training and validation reconstruction (binary cross-entropy) loss of the fully connected autoencoder.

**Required inference:** Determine whether the model converges and whether there is evidence of overfitting.

The plot shows the training and validation binary cross-entropy against the epoch. Both curves fall steeply in the first five epochs and then flatten, because most of the main digit structure is learnt quickly and later epochs only refine details. The two curves lie almost on top of each other so there is no overfitting.

# 9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

### Mean Squared Error

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2.$$

### Mean Absolute Error

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|x_i - \hat{x}_i|.$$

### Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

`https://scikit-image.org/docs/stable/api/skimage.metrics.html`

Report:

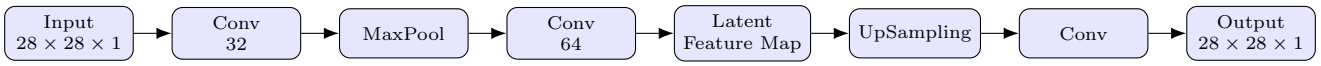
| Metric | Value |
|---|---|
| Test MSE | 0.01996 |
| Test MAE | 0.05433 |
| Mean SSIM | 0.7733 |

The values above are for the fully connected autoencoder (latent dimension 16) on the 2,000 test images.

# 10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:




A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

## 11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

| Model | MSE | MAE | SSIM | Parameters |
|---|---|---|---|---|
| Fully Connected AE | 0.01996 | 0.05433 | 0.7733 | 211,040 |
| Convolutional AE | 0.00248 | 0.01506 | 0.9748 | 74,497 |

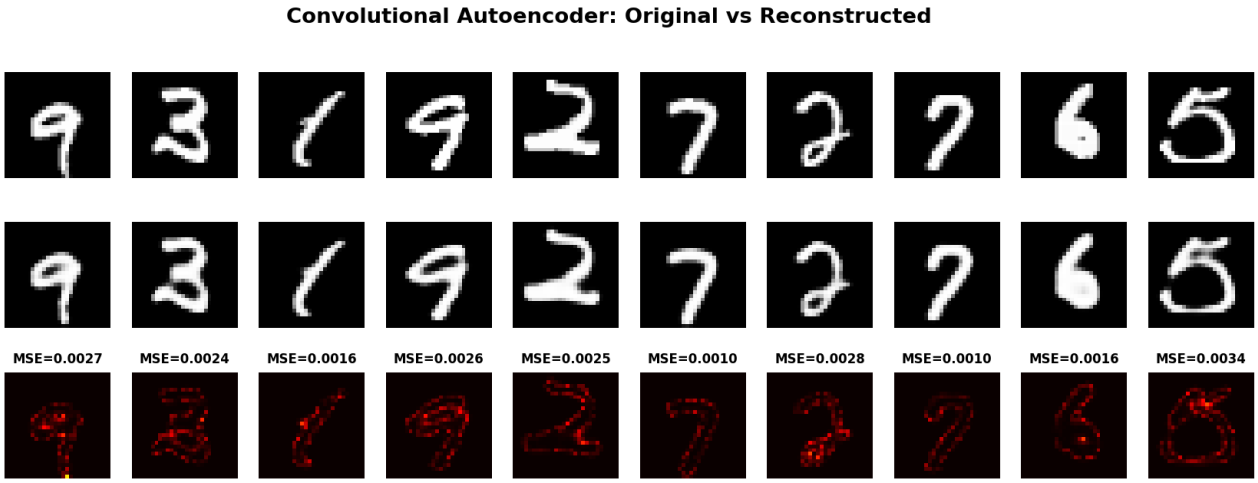



Figure 3: Convolutional autoencoder: original (top), reconstruction (middle) and reconstruction-error map (bottom).

**Observation:** The CAE reconstructions are visibly sharper than the FC-AE above, with digit strokes staying thin and well-defined instead of turning into blobs and the error maps are thinner and concentrated only on stroke edges rather than covering whole digit.

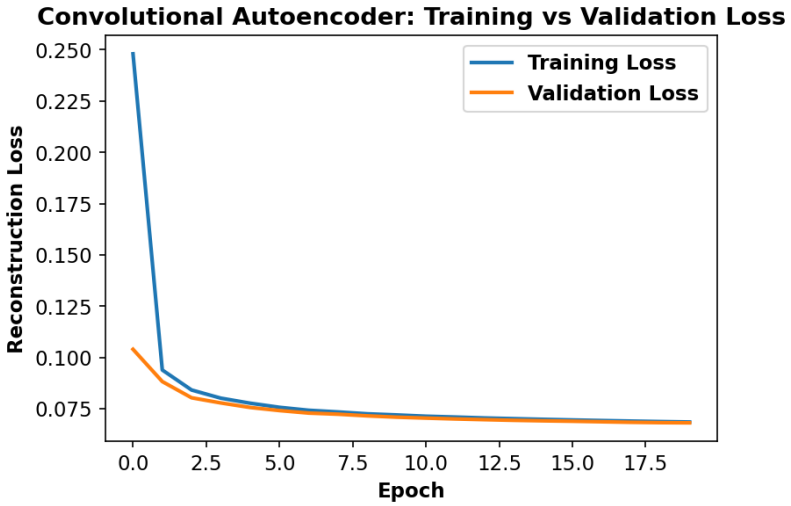



Figure 4: Training and validation loss of the convolutional autoencoder.

**Observation:** Both curves drop sharply in the first two epochs and then flatten smoothly, with the validation loss tracking the training loss closely throughout and never rising above it; so the CAE converges within about 10 epochs with no overfitting.

**Plot 3: Reconstruction Comparison**
Display:

Original → FC-AE Reconstruction → CAE Reconstruction.

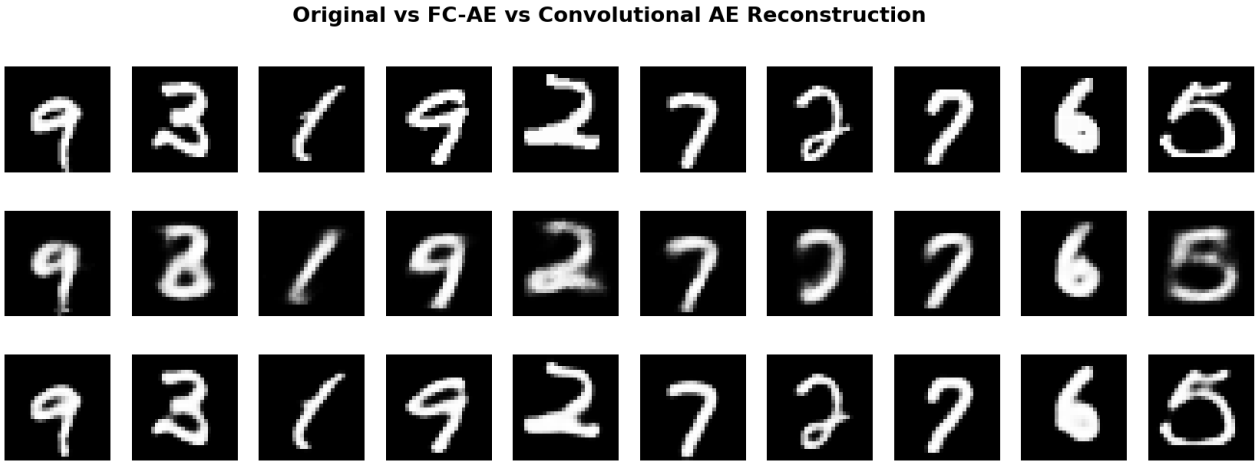



Figure 5: Plot 3: Original (top), FC-AE reconstruction (middle) and CAE reconstruction (bottom).

**Required inference:**
Discuss how preserving spatial locality through convolution affects image reconstruction.

The CAE reconstructs far more sharply than the FC-AE: MSE drops from 0.01996 to 0.00248, MAE from 0.05433 to 0.01506, and SSIM rises from 0.7733 to 0.9748, despite fewer parameters (74,497 vs 211,040). Convolution preserves spatial locality and shares filters across the image, capturing local stroke patterns efficiently, whereas flattening to 784 values discards this. Part of this gain is also because the CAE's bottleneck ($7 \times 7 \times 64$) is larger than the FC-AE's 16-D latent vector, so it compresses less aggressively.

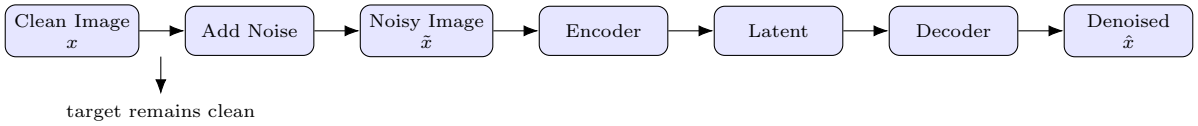
## 12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\boxed{\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where $\tilde{x}$ is a noisy version of the clean image $x$.




# 13. Adding Image Noise

Use at least two corruption types.

## Gaussian Noise

$$\tilde{x} = x + n, \qquad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

## Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.
Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

# 14. Required Denoising Plots

**Plot 4: Clean vs. Noisy vs. Denoised**
For at least 10 images, display:

Clean | Noisy | Denoised.

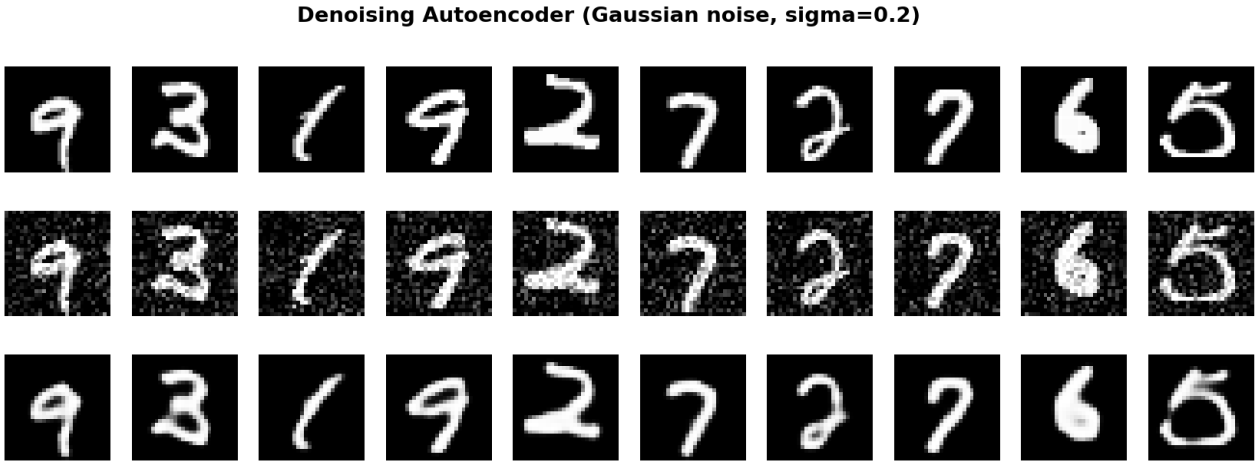



Figure 6: Plot 4(a): Clean (top), noisy (middle, Gaussian $\sigma = 0.2$) and denoised (bottom) test images.

**Observation:** The Gaussian noise makes speckles in the image and partly obscures the strokes, but the denoised row (bottom) recovers clean, smooth digits that are almost same as the originals, showing the denoising CAE has learnt to filter out pixel-level noise while preserving stroke shape.

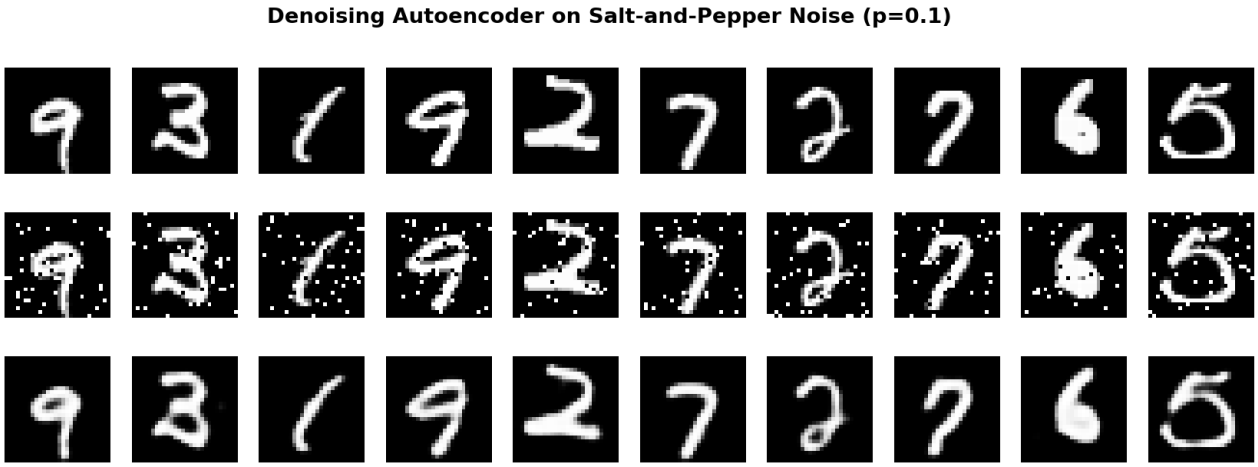



Figure 7: Plot 4(b): Clean (top), noisy (middle, salt-and-pepper $p = 0.10$) and denoised (bottom) test images.

**Observation:** Salt-and-pepper noise corrupts isolated pixels to pure black or white rather than changing all of them slightly, yet the denoised row still restores clean digit strokes with the scattered dots removed.

**Plot 5: Noise Level vs. Reconstruction Quality**

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

| Noise Level | MSE | MAE | SSIM |
|---|---|---|---|
| 0.1 | 0.00352 | 0.01847 | 0.9620 |
| 0.2 | 0.00460 | 0.02141 | 0.9467 |
| 0.3 | 0.00691 | 0.02733 | 0.9172 |

Noise level in the table is the Gaussian standard deviation $\sigma$; a separate denoising CAE was trained and tested at each level. The same for salt-and-pepper noise is:

| Noise Level ($p$) | MSE | MAE | SSIM |
|---|---|---|---|
| 0.05 | 0.00392 | 0.01901 | 0.9581 |
| 0.10 | 0.00512 | 0.02214 | 0.9439 |
| 0.20 | 0.00789 | 0.02855 | 0.9115 |

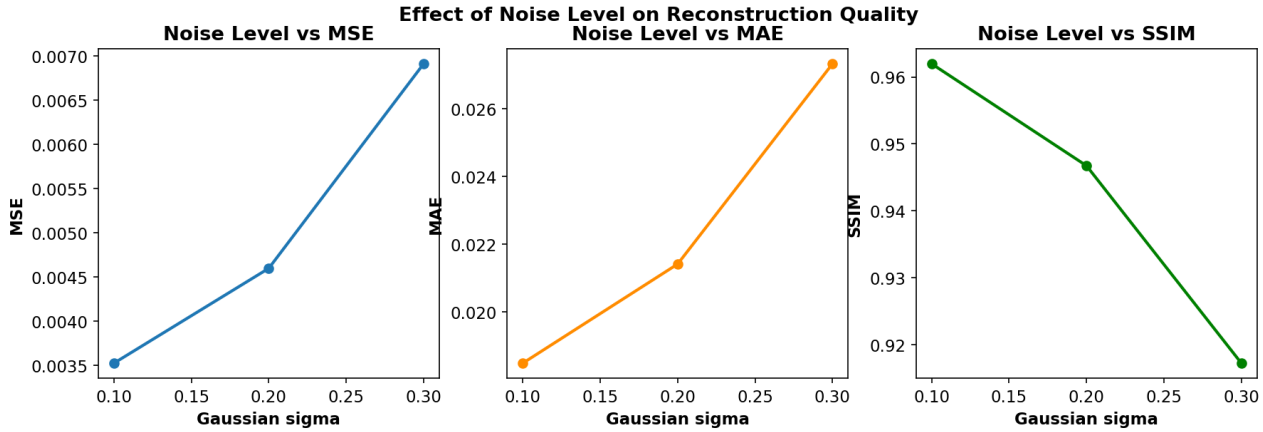



Figure 8: Plot 5: Gaussian noise level $\sigma$ versus MSE, MAE and SSIM (separate panels).

**Observation:** MSE and MAE rise steadily while SSIM falls almost linearly as $\sigma$ increases from 0.1 to 0.3, confirming that heavier Gaussian corruption makes denoising harder, though SSIM stays above 0.91 even at the highest noise level.

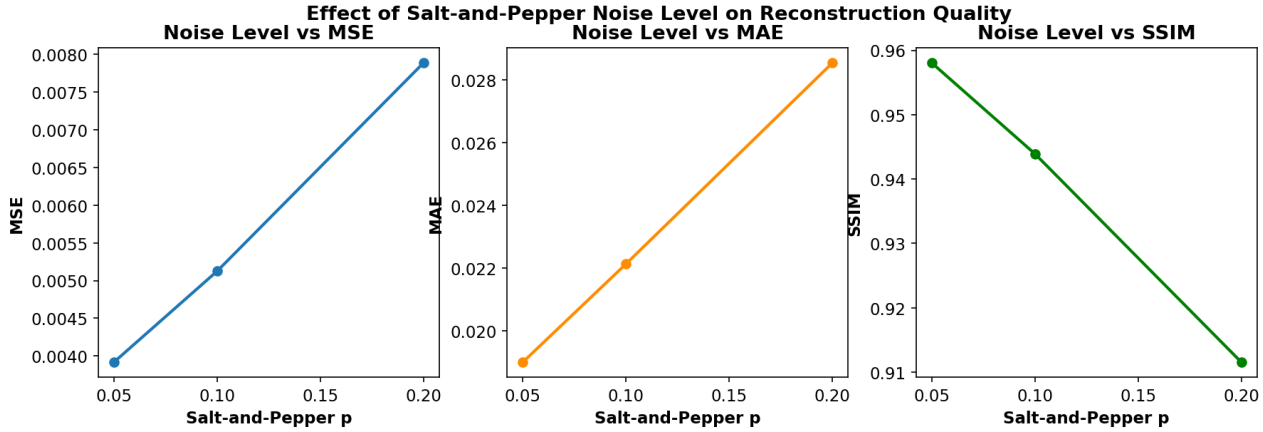



Figure 9: Plot 5(b): Salt-and-pepper noise level $p$ versus MSE, MAE and SSIM (separate panels).

**Observation:** The trend same as the Gaussian case: MSE and MAE increase and SSIM decreases as $p$ rises from 0.05 to 0.20, and the drop is slightly steeper than for Gaussian noise, since salt-and-pepper corruption destroys pixel values outright.

**Required inference:**

Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

Reconstruction quality degrades steadily as corruption increases: Gaussian MSE rises from 0.00352 to 0.00691 and SSIM falls from 0.9620 to 0.9172 as $\sigma$ goes $0.1 \rightarrow 0.3$; salt-and-pepper follows the same trend (SSIM $0.9581 \rightarrow 0.9115$). Stronger noise destroys more stroke-edge information. Even so, the model recovers the major structure of every digit at the highest noise level (SSIM stays above 0.91, denoised images in Plot 4 remain clean and recognisable), since digits lie on a learnable low-dimensional manifold while random noise does not.

# 15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

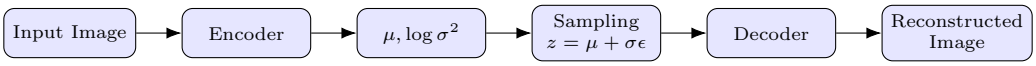
$$q_\phi(z|x) = \mathcal{N}\left(\mu(x), \mathrm{diag}(\sigma^2(x))\right).$$

The encoder produces:

$$\mu, \qquad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$




# 16. VAE Loss Function

The VAE objective consists of two components.

### Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

### KL Divergence

$$D_{KL}\left(q_\phi(z|x) \parallel p(z)\right) = -\frac{1}{2}\sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

# 17. VAE Implementation

Use a small latent space so that it can be visualized.
Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

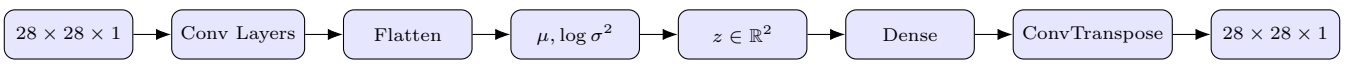
The encoder should produce:

$$\mu \in \mathbb{R}^2, \qquad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \to 28 \times 28 \times 1.$$

A compact architecture is:




# 18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.
**Plot 6: VAE Latent Space**

- X-axis: $z_1$
- Y-axis: $z_2$
- Color/marker category: digit label

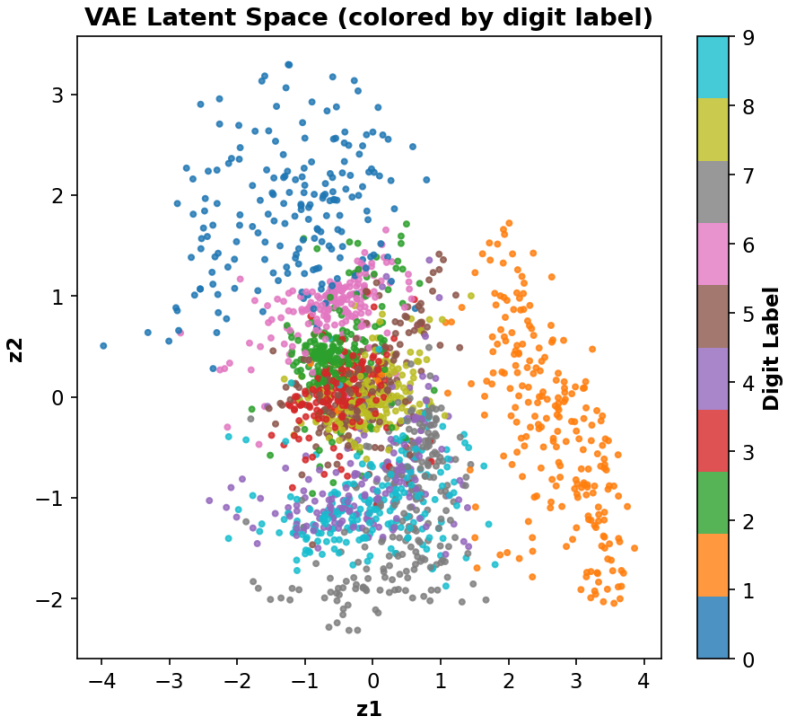



Figure 10: Plot 6: Two-dimensional VAE latent space (encoded test-image means $\mu$), coloured by digit label.

The labels are used only for visualization; they are not supplied to the VAE during training.

**Required inference:**
Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.

The plot shows that similar digits occupy similar regions in the 2-D latent space. Digits 1, 0, 6, and 7 form relatively distinct clusters, while 2, 3, 5, 8 and 4, 9 overlap due to similar shapes and limited latent dimensions. The cloud is centred near the origin because the KL term encourages a Gaussian distribution. Thus, the latent space is continuous and partially digit-organised, but not fully class-separated.

## 19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

**Plot 7: Randomly Generated Images**

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

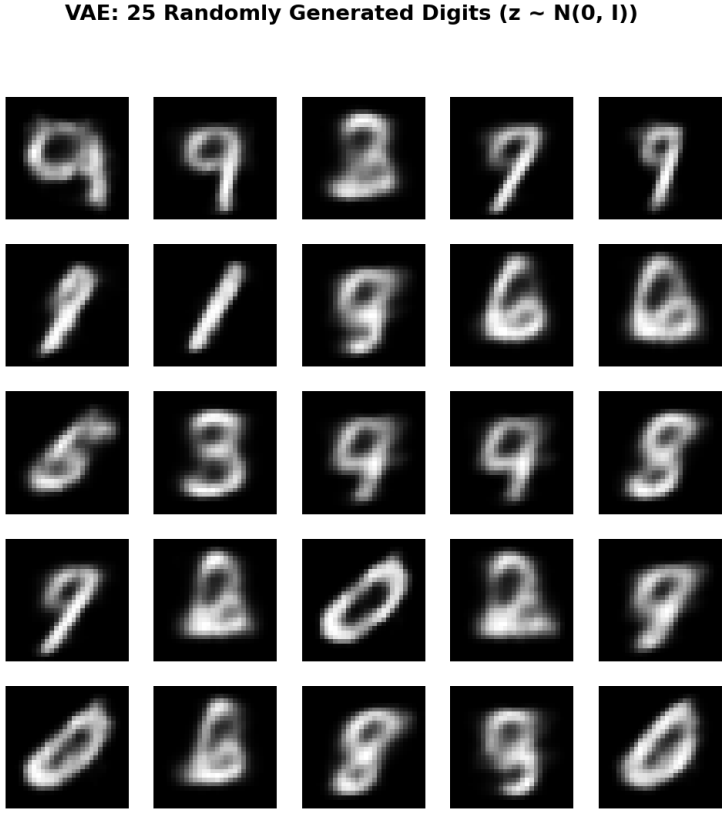



Figure 11: Plot 7: Twenty-five images generated by decoding $z \sim \mathcal{N}(0, I)$.

**Observation:** Most generated images resemble handwritten digits and are recognisable (several 9s, 7s, a 1, 6s, 0s, 3s and 8s), and they vary in class, slant and thickness, so the generator is not producing a single repeated shape. However, all samples are blurry, and a few are ambiguous blends between neighbouring classes (for example blob-like shapes between 2, 3 and 8, or 9-like shapes with a partly open loop); no clear 4 appears in this particular set.

## 20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \qquad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \qquad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0,\ 0.1,\ 0.2, \ldots, 1.$$

Decode every interpolated vector.

**Plot 8: Latent Space Interpolation**

Display the decoded images in sequence.

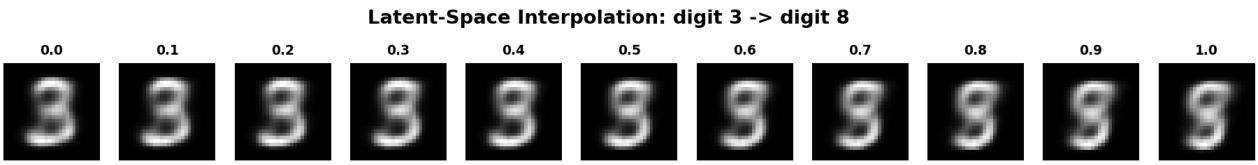



Figure 12: Plot 8: Decoded images along $z(\alpha)$ for $\alpha = 0, 0.1, \ldots, 1$, from a test digit 3 to a test digit 8.

**Required inference:**

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

The decoded image changes smoothly and gradually from the 3 to the 8 as $\alpha$ increases, with no abrupt jump. The change is subtle because the two chosen points lie close together in the overlapping central region of the latent space. This smoothness is a direct result of the KL regularisation, which makes the latent space continuous so that nearby points decode to similar images; it shows that the VAE has learnt a meaningful, continuous representation rather than memorising isolated points.

## 21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

| Model | MSE | MAE | SSIM | Parameters |
|---|---|---|---|---|
| Fully Connected AE | 0.01996 | 0.05433 | 0.7733 | 211,040 |
| Convolutional AE | 0.00248 | 0.01506 | 0.9748 | 74,497 |
| Denoising CAE | 0.00460 | 0.02141 | 0.9467 | 74,497 |

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

| **VAE Metric** | **Value** |
|---|---|
| Reconstruction Loss | 152.362 |
| KL Loss | 5.692 |
| Total Loss | 158.054 |
| Test MSE | 0.04492 |
| Test MAE | 0.10270 |
| Mean SSIM | 0.5058 |
| Parameters | 134165 |

Table 1: VAE Performance Metrics

The Denoising CAE is evaluated on Gaussian noise $\sigma = 0.2$ (denoised output against the clean image); the VAE uses a 2-D latent space, and its reconstruction and KL losses are summed over pixels and latent dimensions respectively.

## 22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

All ten plots are included in the sections above: Plot 1 (Sec. 8), Plot 2 (Sec. 8), Plot 3 (Sec. 11), Plot 4 and Plot 5 (Sec. 14), Plot 6 (Sec. 18), Plot 7 (Sec. 19), Plot 8 (Sec. 20), the VAE loss plot (below) and the error distribution (Sec. 23).

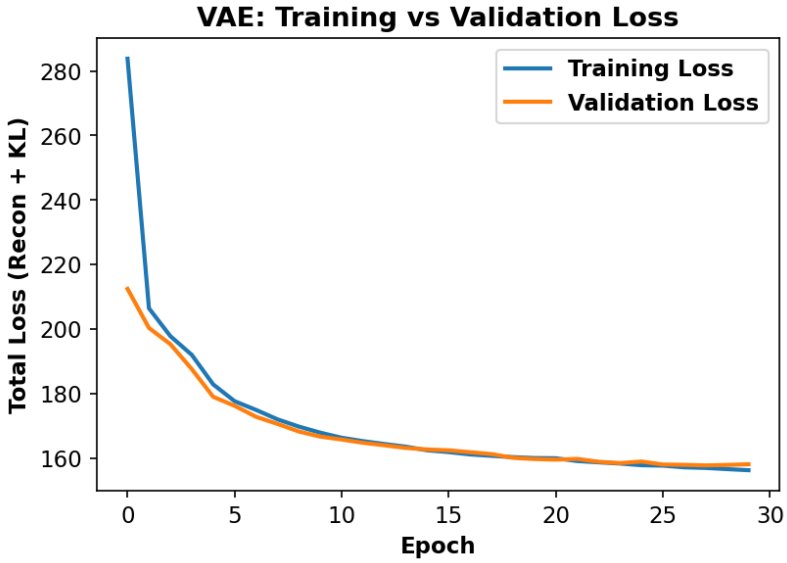



Figure 13: Training and validation loss (reconstruction + KL) of the VAE.

**Inference:** The VAE training loss falls from about 284 to 156 and the validation loss from about 212 to 158; the sharp early drop reflects the decoder quickly learning the average digit appearance, after which both curves flatten. The two curves almost coincide at the end , so the VAE converged without overfitting.

## 23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D}\sum_{j=1}^{D}(x_{ij} - \hat{x}_{ij})^2.$$

**Plot 9: Distribution of Reconstruction Errors**

Plot a histogram of $e_i$ over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

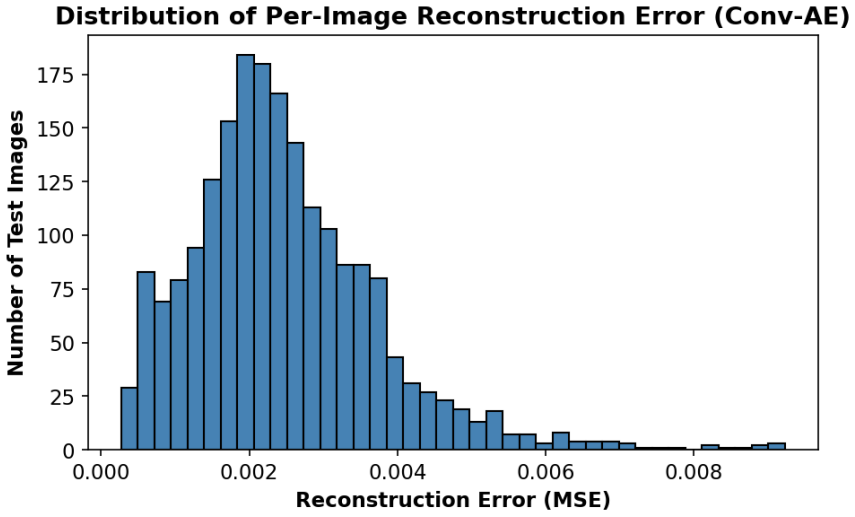



Figure 14: Plot 9: Histogram of per-image reconstruction error $e_i$ on the test set (convolutional autoencoder).

**Observation:**The convolutional autoencoder has a mean per-image MSE of 0.00248, with most errors between 0.001 and 0.004. The distribution is right-skewed, with a few high-error images showing noticeably blurrier or distorted reconstructions, while most images are reconstructed closely.

## 24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.
Display:

| Sample | Original | Reconstruction |
|---|---|---|
| 1 | test image idx 892, $e = 0.0092$ | see figure |
| 2 | test image idx 152, $e = 0.0092$ | see figure |
| 3 | test image idx 949, $e = 0.0090$ | see figure |
| 4 | test image idx 287, $e = 0.0090$ | see figure |
| 5 | test image idx 1980, $e = 0.0089$ | see figure |

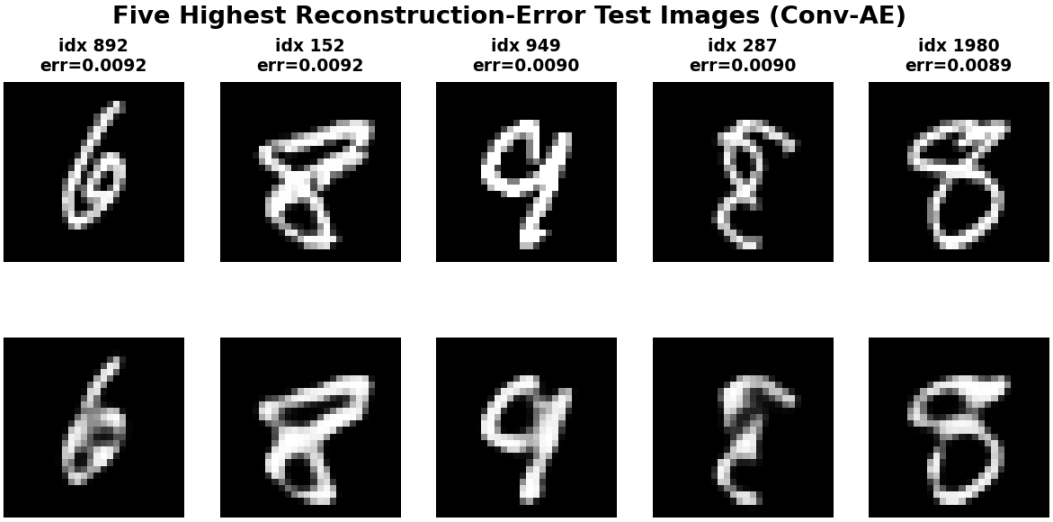



Figure 15: The five test images with the largest reconstruction error (top: original, bottom: reconstruction).

Students should provide a short explanation for why these images may be difficult to reconstruct.

These images are difficult to reconstruct because they are written in an atypical way: heavy slant, extra loops and strokes crossing each other (the three 8-like samples and the 6-like sample), an unusually thin stroke, or a shape (the 9/4-like sample) that resembles another class. The decoder reproduces the common average shape of the digit and therefore smooths or drops the unusual details, which produces the largest pixel-wise error. So reconstruction error is related to visual quality: the low-error images look almost identical to the original, while the high-error ones show visibly distorted or blurred strokes.

## 25. Consolidated Results

| Model | MSE | MAE | SSIM | Parameters | Time |
|---|---|---|---|---|---|
| FC Autoencoder | 0.01996 | 0.05433 | 0.7733 | 211,040 | 13.5 s |
| Conv. Autoencoder | 0.00248 | 0.01506 | 0.9748 | 74,497 | 24.2 s |
| Denoising CAE | 0.00460 | 0.02141 | 0.9467 | 74,497 | 16.2 s |
| VAE | 0.04492 | 0.10270 | 0.5058 | 134,165 | 31.5 s |

The values must be obtained from the student's own execution.
Time is the training time on a Colab GPU (20 epochs for FC-AE and CAE, 15 for the denoising CAE, 30 for the VAE), so it is not directly comparable across models. The Denoising CAE row uses Gaussian noise $\sigma = 0.2$.

# 26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

**Inferences for the mandatory items** (each plot's own inference is given next to the plot in the earlier sections):

1. **Effect of latent dimension on reconstruction.** The table and plot in Section 27 show MSE falling from 0.05012 ($d_z = 2$) to 0.02074 ($d_z = 32$) and SSIM rising from 0.4338 to 0.7615. A very small latent vector cannot store enough information, whereas a larger one keeps more detail; the gains shrink at large $d_z$ (MSE $0.02287 \to 0.02074$ from 16 to 32), so there are diminishing returns.
2. **Fully connected vs. convolutional reconstruction.** The CAE gives about $8\times$ lower MSE and SSIM 0.9748 against 0.7733 with fewer parameters, because convolution exploits spatial locality and weight sharing (Section 11).
3. **Effect of Gaussian noise level.** MSE rises ($0.00352 \to 0.00691$) and SSIM falls ($0.9620 \to 0.9172$) as $\sigma$ increases from 0.1 to 0.3, because stronger noise hides more of the stroke information (Section 14).
4. **Ability of the denoising autoencoder to recover structure.** Even at $\sigma = 0.3$ SSIM stays above 0.91 and the denoised digits are clean and recognisable, since the network learns the digit manifold and rejects independent noise.
5. **Characteristics of the VAE latent distribution.** The encoded means form a single, roughly Gaussian cloud centred near the origin without large gaps (Plot 6), with a small KL loss of 5.69 nats, showing that the KL term keeps the posterior close to $\mathcal{N}(0, I)$.
6. **Structure of the two-dimensional latent space.** It is partly organised by class (1 on the right, 0 upper left, 7 at the bottom, 6 upper middle) but 2/3/5/8 and 4/9 overlap heavily, because ten classes are hard to separate in two unsupervised dimensions.
7. **Quality and diversity of generated samples.** Samples are recognisable and diverse in class and style but blurry, and some are ambiguous blends (Plot 7 and the $10 \times 10$ grid in Section 29).
8. **Smoothness of latent interpolation.** Both interpolations (3→8 and 1→7) change gradually with no abrupt jumps, showing a continuous latent space.
9. **Relationship between reconstruction error and visual quality.** Low-error images are almost identical to their originals, while the highest-error images (Section 24) are unusual, slanted or overlapping strokes that come out blurred or distorted; the CAE error histogram is concentrated at $\approx 0.002$ with a long right tail.
10. **Deterministic vs. probabilistic latent representations.** The FC-AE, CAE and denoising CAE map each image to one fixed code $z = f(x)$ and reconstruct best (CAE SSIM 0.9748 against VAE 0.5058), but they are not constrained to any prior and cannot generate new images by sampling. The VAE encodes a distribution $q(z|x)$, is regularised by the KL term towards $\mathcal{N}(0, I)$, which costs reconstruction accuracy (especially with a 2-D latent space) but gives a smooth latent space that supports sampling and interpolation.

# 27. Source Code

GitHub Repository

# 28. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

| Latent Dimension | MSE | SSIM | Parameters |
|---|---|---|---|
| 2 | 0.05012 | 0.4338 | 210,130 |
| 8 | 0.02792 | 0.6936 | 210,520 |
| 16 | 0.02287 | 0.7384 | 211,040 |
| 32 | 0.02074 | 0.7615 | 212,080 |

**Plot 10: Latent Dimension vs. Reconstruction MSE**

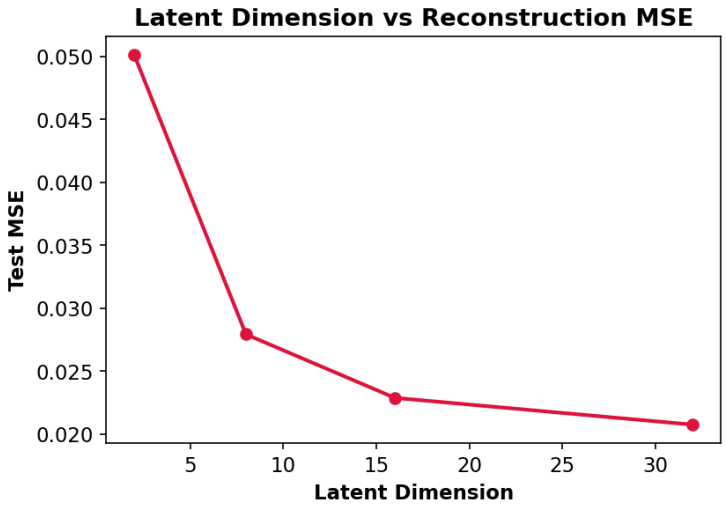



Figure 16: Plot 10: Latent dimension versus test reconstruction MSE (fully connected autoencoder).

Students should explain the trade-off between compression and reconstruction quality.

As the latent dimension increases, reconstruction error decreases and image quality improves. However, the improvement becomes smaller at higher dimensions. Therefore, a latent dimension of 16 provides a good balance between compression and reconstruction quality, while the parameter count remains nearly unchanged.

## 29. Discussion Questions

1. What is an autoencoder?
   **Answer:** An autoencoder is a neural network trained to reproduce its own input at the output through a lower-dimensional bottleneck, so it learns a compressed representation of the data without labels.
2. What is the purpose of the encoder?
   **Answer:** The encoder maps the high-dimensional input $x$ to a lower-dimensional latent representation $z = f_\theta(x)$, keeping the information needed for reconstruction.
3. What is the purpose of the decoder?
   **Answer:** The decoder reconstructs the input from the latent representation, $\hat{x} = g_\phi(z)$.
4. What is a latent representation?
   **Answer:** A latent representation is the compressed vector (or feature map) $z$ produced by the encoder that captures the essential features of the input.
5. Why is the latent representation usually smaller than the input?
   **Answer:** It is made smaller so that the network is forced to learn a compact, meaningful encoding instead of copying the input (identity mapping).
6. What happens when the latent dimension is too small?
   **Answer:** The bottleneck cannot hold enough information, so reconstructions are blurry and the error is high (e.g. MSE 0.05012 and SSIM 0.4338 for $d_z = 2$).
7. What happens when the latent dimension is increased?
   **Answer:** Reconstruction improves (lower MSE, higher SSIM), with diminishing returns, but compression is reduced and the model moves closer to an identity mapping (MSE 0.02074 at $d_z = 32$).

8. Why can a convolutional autoencoder preserve spatial information more effectively?
   **Answer:** Convolutions work on local neighbourhoods and share filters over the image, so the 2-D spatial layout is retained, whereas flattening loses it; the CAE reached SSIM 0.9748 against 0.7733 for the FC-AE.
9. Differentiate an autoencoder and a denoising autoencoder.
   **Answer:** A plain autoencoder receives a clean image and reconstructs the same image, whereas a denoising autoencoder receives a corrupted image $\tilde{x}$ and must reconstruct the clean $x$, so it learns to remove noise.
10. Why is the clean image used as the target in denoising?
    **Answer:** If the noisy image were the target, the network would learn to reproduce the noise; using the clean target forces it to learn the underlying structure and discard the corruption.
11. What happens when the noise level is increased?
    **Answer:** The corruption destroys more information, so MSE and MAE increase and SSIM decreases (MSE $0.00352 \to 0.00691$ and SSIM $0.9620 \to 0.9172$ for $\sigma = 0.1 \to 0.3$).
12. Why are MSE, MAE and SSIM useful for image reconstruction?
    **Answer:** MSE penalises large pixel errors, MAE gives the average absolute error and is less sensitive to outliers, and SSIM measures structural (luminance, contrast, structure) similarity that is closer to human perception; together they give complementary views of quality.
13. What is a Variational Autoencoder?
    **Answer:** A VAE is a generative autoencoder whose encoder outputs the parameters $(\mu, \log \sigma^2)$ of a distribution over the latent space, from which $z$ is sampled and decoded, trained with a reconstruction loss plus a KL-divergence term.
14. How does a VAE differ from a conventional autoencoder?
    **Answer:** A conventional autoencoder gives one deterministic code $z = f(x)$ and has no constraint on the latent space, whereas a VAE gives a distribution, uses sampling with the reparameterisation trick and is regularised towards a prior.
15. Why does a VAE learn a distribution instead of a single latent vector?
    **Answer:** Learning a distribution makes the latent space continuous and smooth, so nearby points decode to similar images and new points can be sampled and decoded to generate meaningful images.
16. What is the reparameterization trick?
    **Answer:** It writes the sample as $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, moving the randomness into $\epsilon$ so that gradients can flow through $\mu$ and $\sigma$ during backpropagation.
17. Why is $\epsilon$ sampled from a standard normal distribution?
    **Answer:** The standard normal has zero mean and unit variance, so it is a simple, fixed noise source independent of the parameters, and $\sigma \odot \epsilon$ then scales it to the desired variance; it also matches the prior $\mathcal{N}(0, I)$.
18. What is the role of KL divergence in the VAE loss?
    **Answer:** The KL term pushes each $q_\phi(z|x)$ towards the prior $\mathcal{N}(0, I)$, keeping the latent space compact, continuous and samplable; without it the VAE would behave like a plain autoencoder with isolated codes.
19. What is the prior distribution used in the VAE?
    **Answer:** The prior is the standard normal distribution, $p(z) = \mathcal{N}(0, I)$.
20. Why is a two-dimensional latent space useful for visualization?
    **Answer:** With two dimensions the encoded test images can be plotted directly in a plane (Plot 6) to inspect clustering, overlap and structure without dimensionality reduction.
21. What does latent-space interpolation demonstrate?
    **Answer:** It shows that the latent space is continuous and smooth: moving between two latent points gives a gradual, meaningful morph between the two images (Plots 8 and Exercise 8).
22. Why can a VAE generate new images?
    **Answer:** Its latent space is regularised to follow $\mathcal{N}(0, I)$, so sampling $z \sim \mathcal{N}(0, I)$ and passing it through the decoder yields new images that resemble the training data.
23. Why might some VAE-generated images be unclear?
    **Answer:** Sampled points may fall between class regions or in low-density areas, the decoder outputs an average of nearby images, and the reconstruction term is a pixel-wise loss, so the images are blurry or ambiguous blends (e.g. between 2, 3 and 8).
24. What is the difference between reconstruction and generation?
    **Answer:** Reconstruction encodes a real image and decodes its own code to approximate that image, whereas generation samples a latent vector from the prior and decodes it to create a new image with no corresponding input.
25. Why should test images remain unseen during training?
    **Answer:** Unseen test images give an unbiased estimate of generalisation; if the model saw them during training, the metrics would measure memorisation and could hide overfitting.

## 30. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.

2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

## Results of the Additional Exercises

**Exercise 1: Convolutional autoencoder with latent dimensions 4, 8, 16, 32.** A dense bottleneck of the given size was inserted between the convolutional encoder and decoder (15 epochs).

| Latent Dimension | MSE | MAE | SSIM | Parameters |
|---|---|---|---|---|
| 4 | 0.04214 | 0.09814 | 0.5471 | 65,797 |
| 8 | 0.02854 | 0.07092 | 0.6978 | 90,889 |
| 16 | 0.02194 | 0.05784 | 0.7668 | 141,073 |
| 32 | 0.01267 | 0.03883 | 0.8654 | 241,441 |

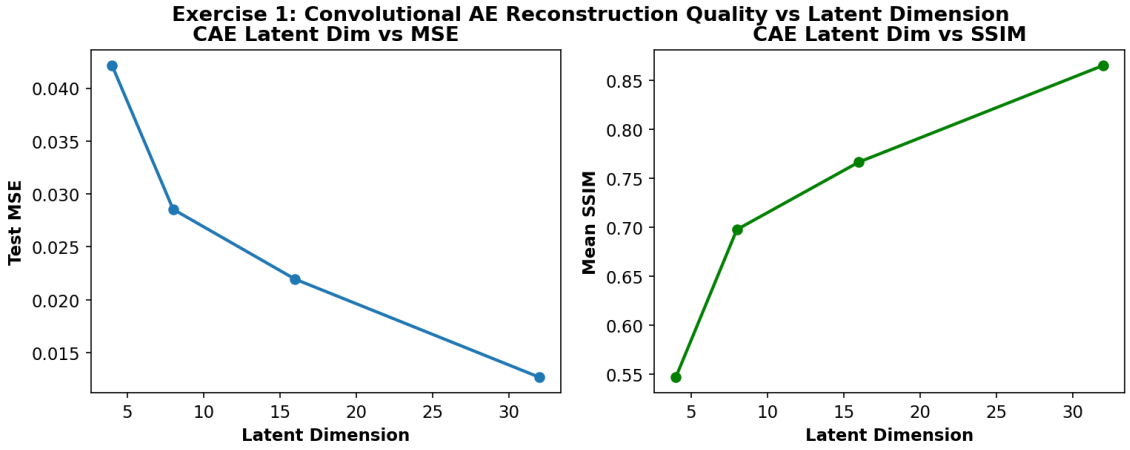



Figure 17: Exercise 1: CAE latent dimension versus MSE (left) and SSIM (right).

Reconstruction quality improves steadily with latent size (MSE 0.04214 → 0.01267, SSIM 0.5471 → 0.8654), because a larger code retains more detail. Unlike the FC-AE study, the gain from 16 to 32 is still large. With the same latent size the CAE is comparable to the FC-AE at small sizes but clearly better at 32 (MSE 0.01267 against 0.02074), and the 7×7×64 feature-map CAE of Section 10 (0.00248) is far better because it does not compress into a small vector.

**Exercise 2: Gaussian vs. salt-and-pepper noise (same architecture).** Using the same denoising CAE trained at Gaussian $\sigma = 0.2$ and salt-and-pepper $p = 0.10$:

| Noise type | MSE | MAE | SSIM |
|---|---|---|---|
| Gaussian ($\sigma = 0.2$) | 0.00460 | 0.02141 | 0.9467 |
| Salt-and-pepper ($p = 0.10$) | 0.00512 | 0.02214 | 0.9439 |

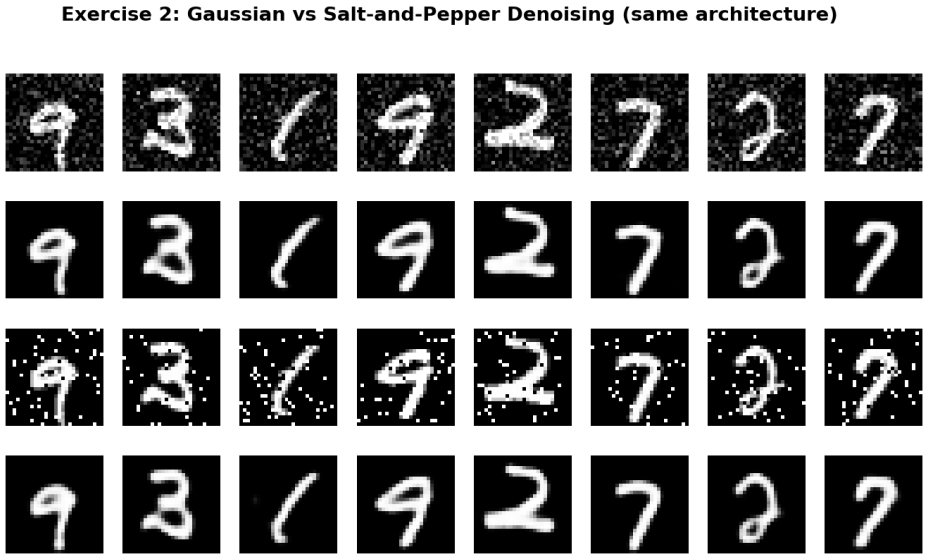



Figure 18: Exercise 2: Noisy inputs and denoised outputs for Gaussian and salt-and-pepper noise.

Both corruptions are removed well and the metrics are close; salt-and-pepper noise is slightly harder (MSE 0.00512 against 0.00460, SSIM 0.9439 against 0.9467) because it replaces pixels outright by 0 or 1 instead of changing them slightly, so the true value of the corrupted pixels must be inferred from their neighbours. The two settings are not exactly matched in amount of corruption, so the comparison is approximate.

**Exercise 3: Two noise levels compared by SSIM.** The denoising CAE trained at $\sigma = 0.1$ reaches SSIM 0.9620 and the one at $\sigma = 0.3$ reaches 0.9172, a drop of 0.0447. Higher noise makes denoising harder, but even at $\sigma = 0.3$ the model still recovers the digits well (Section 14).

**Exercise 4: Transposed-convolution decoder.**

| Decoder | MSE | MAE | SSIM | Parameters |
|---|---|---|---|---|
| UpSampling2D (original) | 0.00248 | 0.01506 | 0.9748 | 74,497 |
| Conv2DTranspose | 0.00263 | 0.01529 | 0.9740 | 74,497 |

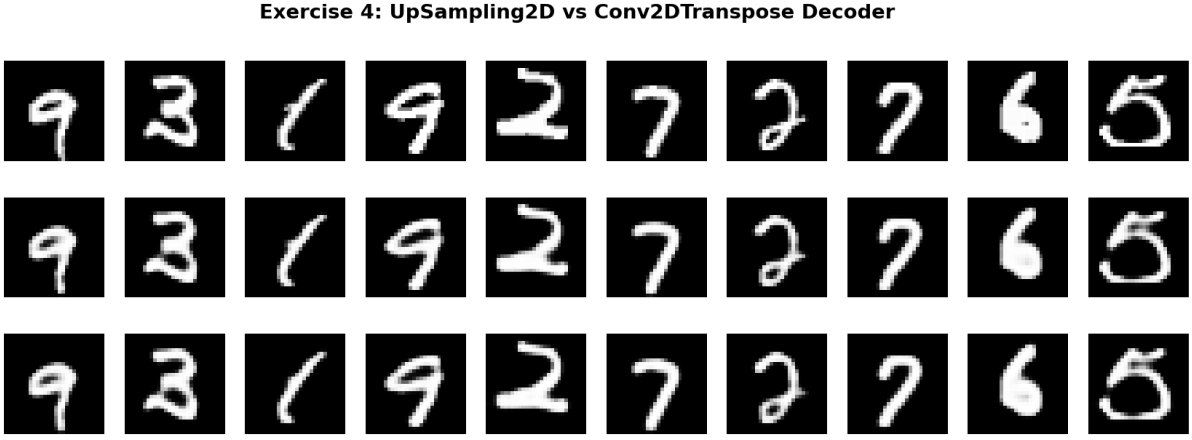



Figure 19: Exercise 4: Original, UpSampling2D reconstruction and Conv2DTranspose reconstruction.

The two decoders give almost identical reconstructions (SSIM 0.9748 against 0.9740, the same parameter count). The transposed convolution learns its own upsampling filters, but here it brings no measurable improvement (MSE slightly higher, 0.00263 against 0.00248), so for this small MNIST model the choice of decoder is not important.

**Exercise 5: VAE with latent dimension 2 and 8.**

| VAE | Recon. Loss | KL Loss | MSE | SSIM |
|---|---|---|---|---|
| $d_z = 2$ | 152.362 | 5.692 | 0.04492 | 0.5058 |
| $d_z = 8$ | 119.552 | 12.660 | 0.03040 | 0.6830 |

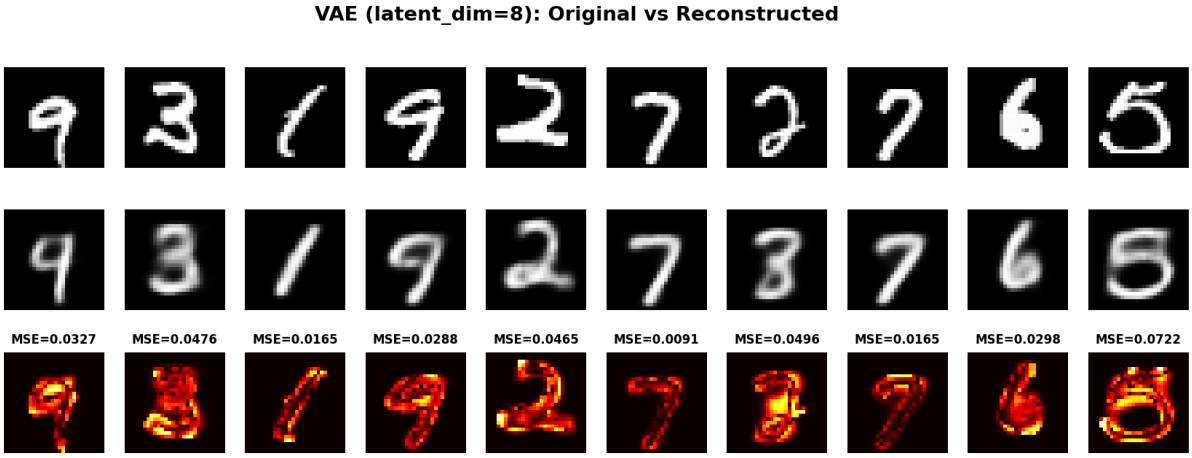



Figure 20: Exercise 5: Original, reconstruction and error map for the VAE with latent dimension 8.

The 8-D VAE reconstructs much better (reconstruction loss 152.4 → 119.6, SSIM 0.5058 → 0.6830) because it can store more information, and its KL loss is larger (5.69 → 12.66) since more latent dimensions are used to encode each image. The 2-D latent space can be plotted and inspected directly but is heavily overlapped, while the 8-D latent space is richer but cannot be visualised directly.

**Exercise 6: Effect of increasing the KL-loss contribution ($\beta$-VAE).** The loss was changed to $\mathcal{L}_{rec} + \beta\mathcal{L}_{KL}$ (20 epochs, $d_z = 2$); the KL value reported is the weighted term $\beta D_{KL}$.

| $\beta$ | Recon. Loss | Weighted KL | MSE | SSIM |
|---|---|---|---|---|
| 1 | 160.572 | 5.730 | 0.04877 | 0.4578 |
| 5 | 182.508 | 8.093 | 0.05792 | 0.3257 |

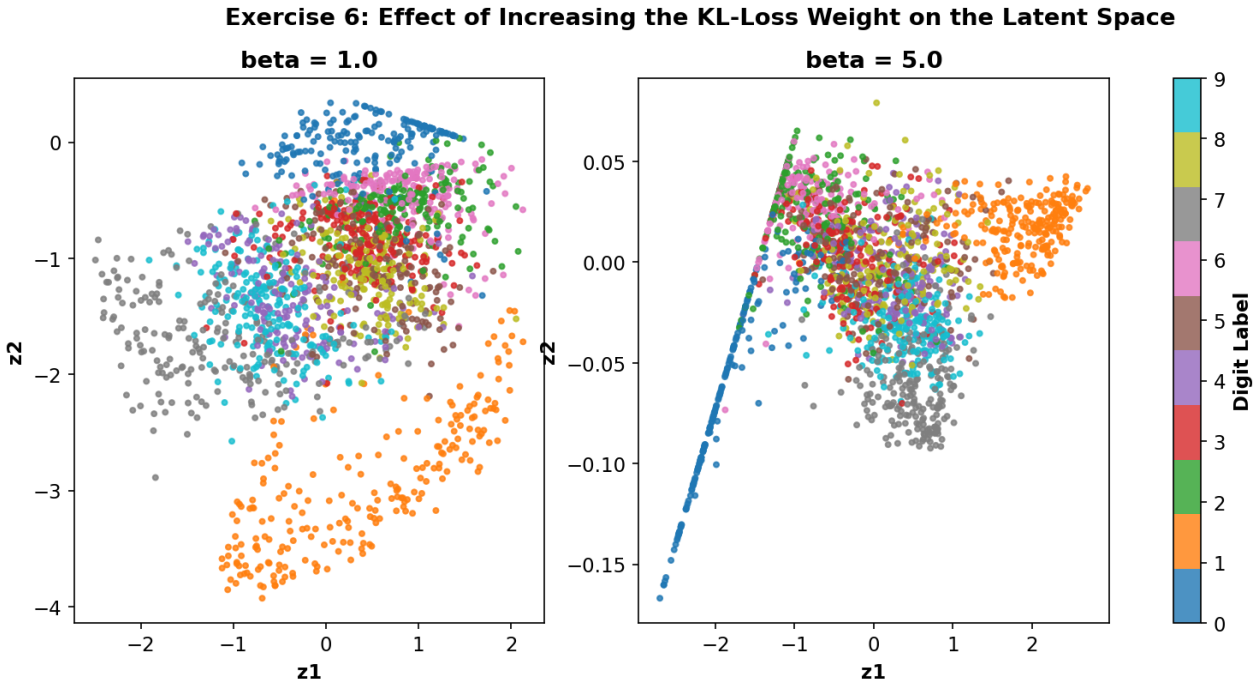



Figure 21: Exercise 6: VAE latent space for $\beta = 1$ (left) and $\beta = 5$ (right).

Increasing beta strengthens the prior, causing higher reconstruction loss and lower SSIM. The latent codes become more compressed, with one dimension nearly unused and greater overlap between digit classes. Thus, a larger KL weight produces a more regular latent space but poorer reconstruction quality.

**Exercise 7: Larger grid of VAE samples.** One hundred images were generated from $z \sim \mathcal{N}(0, I)$.

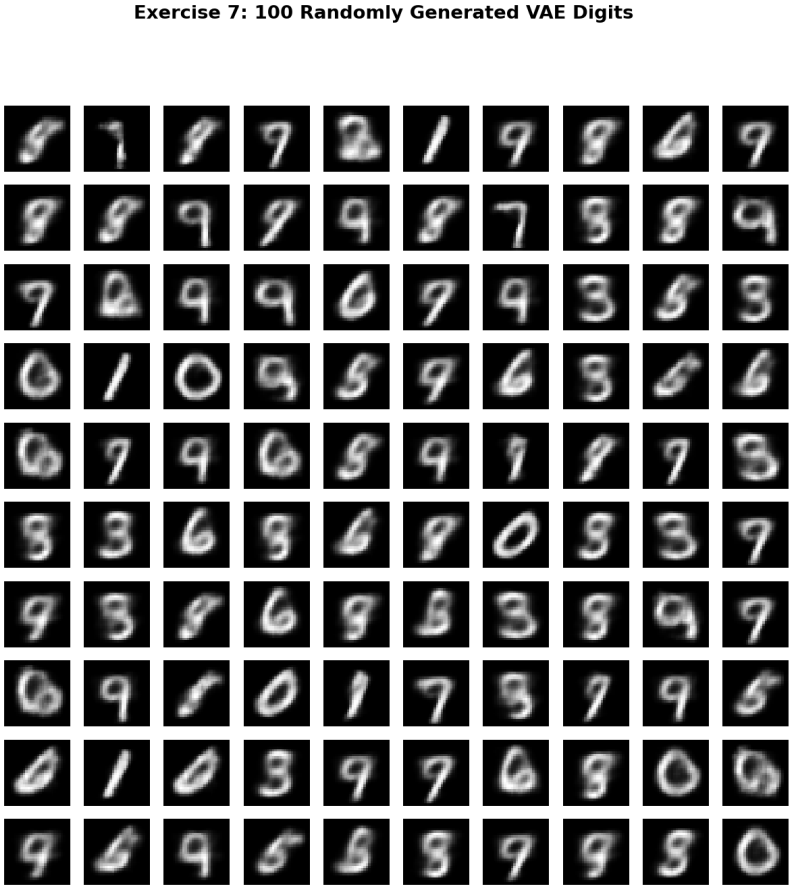



Figure 22: Exercise 7: One hundred randomly generated VAE images.

The larger grid shows good diversity: 0, 1, 3, 5, 6, 7, 8 and 9 appear with different slants and thicknesses (the mean pairwise pixel difference among 30 samples is 0.1003). The classes are unevenly represented, with many 9, 7 and 8 like samples, and many images are blurry or ambiguous blends, again because sampled points can fall between class regions of the 2-D latent space.

**Exercise 8: Interpolation between two digit regions.** Latent means of a test digit 1 (right cluster) and a test digit 7 (bottom cluster) were interpolated (the 3→8 case is Plot 8).

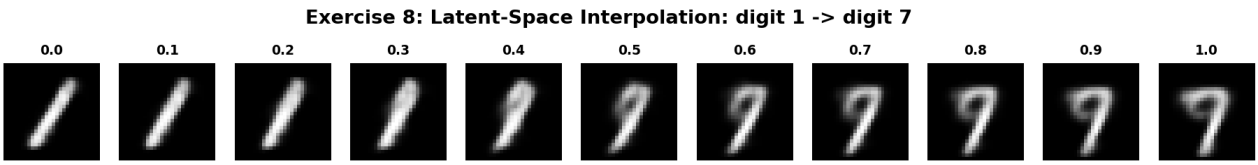



Figure 23: Exercise 8: Decoded images for interpolation from a digit 1 to a digit 7 ($\alpha = 0, 0.1, \ldots, 1$).

The image changes gradually: the slanted stroke of the 1 slowly acquires the horizontal top bar of the 7, with blurry intermediate shapes around $\alpha \approx 0.4$–$0.5$. This transition is more visible than the 3→8 case because the two digits lie in well-separated regions of the latent space, and it confirms the smoothness of the learnt latent space.

# 31. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

$$\boxed{\text{Image} \rightarrow \text{Encoder} \rightarrow \text{Latent Representation} \rightarrow \text{Decoder} \rightarrow \text{Reconstructed Image}}$$

They should also demonstrate denoising:

$$\boxed{\text{Clean Image} \rightarrow \text{Noise} \rightarrow \text{Denoising Autoencoder} \rightarrow \text{Recovered Image}}$$

and variational generation:

$$\boxed{\text{Image} \rightarrow (\mu, \sigma) \rightarrow z \rightarrow \text{Decoder} \rightarrow \text{Generated/Reconstructed Image}}$$

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

The complete pipeline was demonstrated on MNIST: the convolutional autoencoder reconstructs images with SSIM 0.9748, the denoising autoencoder recovers clean digits from noisy ones (SSIM 0.9467 at $\sigma = 0.2$), and the VAE generates new digits by sampling the latent space and interpolates smoothly between digits. Quantitatively, the convolutional model gives the best reconstruction, while only the VAE provides generation, at the cost of reconstruction accuracy.

# 32. References


1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`
7. scikit-image Documentation: `https://scikit-image.org`